Glaze Search: the pretrained models

Every pretrained model in the app, step by step: what it is, where it is used, and whether it is used zero-shot, frozen with a trained head, or fine-tuned. With a picture guide: one real photo and one real sentence followed through every model. Checked in the code (this repository) on 5 November 2026.

The short answer

5 pretrained models run in the app: **Grounding DINO** (base), **SAM 2.1** (large), **OpenCLIP ViT-B/32**, **DINOv2 ViT-L/14** and **all-MiniLM-L6-v2**.

None of them is fine-tuned in the app. Each one is used in one of two ways:

ZERO-SHOT as it is, with text prompts (no training on our data), or

FROZEN + TRAINED HEAD frozen: its weights never change, it only turns an image or a text into a vector; a **small new layer is trained** on our data on top of that vector (transfer learning, not fine-tuning).

Fine-tuning was **tried once** as an experiment (Grounding DINO's decoder on the six attributes, *mm/dino_finetune.py*); the app does not use it.

"So no fine-tuning: only prompts, and some numbers adjusted?"

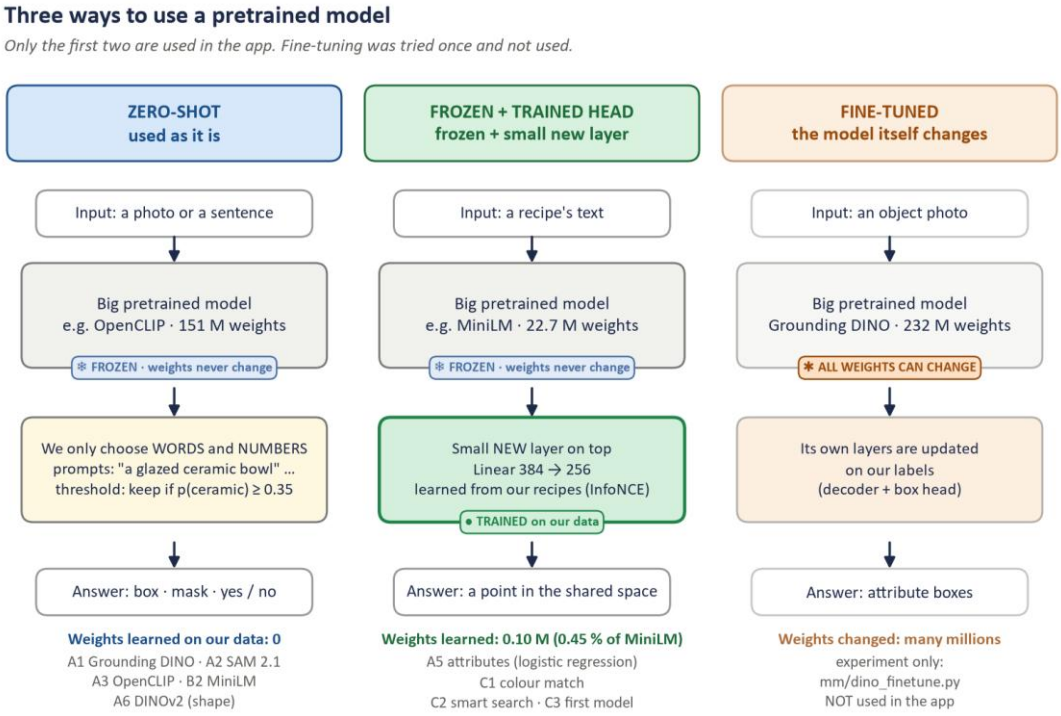


Figure 1 · The three ways a pretrained model can be used. The app uses the first two; fine-tuning was only an experiment.

Partly right. There are three different things we adjust, and only one of them is learning:

1 · Prompts (words). We write what the model should look for: *"ceramic bowl. pottery shard …"*, *"a glazed ceramic bowl"*, *"mirror-like reflection"*. **The model is not changed.**

2 · Cut-offs and weights (numbers). We choose them by hand and check them on development photos or validation queries: keep an object if p(ceramic) ≥ 0.35, colour match = 0.40 learned + 0.40 colour + 0.10

surface + 0.10 cone. **Still not learning.**

3 · Small trained layers (real learning). For A5, C1, C2 and C3 a **new small layer is trained** on our data with gradient descent (for example 98,560 + 4,352 weights in the colour-match model). The big model under it stays frozen. This is transfer learning, **not fine-tuning**: fine-tuning would change the big model's own millions of weights.

What is adjusted	Real values in the app	Steps	Learned from data?
Prompts (words)	"ceramic object. pottery piece. … ceramic bowl. glazed pottery." · "human hand. finger." · 30 CLIP sentences (12 ceramic, 18 not) · 2 phrases per attribute value	A1, A3, A5	no: written by us
Cut-offs	Grounding DINO 0.15 (find everything first) · hand boxes 0.35 · CLIP p(hand) ≥ 0.5 · p(ceramic) ≥ 0.35 · attribute "sure" = the cut-off that was ≥ 98 % right on validation	A1 – A5	no: chosen and checked on development photos
Score weights	colour match 0.40 / 0.40 / 0.10 / 0.10 · smart search text 0.35, photo 0.35, formula 0.25, colour 0.30, cone 0.10, surface 0.10, transparency 0.05, atmosphere 0.05 (renormalised)	A6, C1, C2	no: set in the config files (A6: picked on validation queries)
Small new layers	Linear 384→256 (98,560) + Linear 16→256 (4,352) · 3 MLP heads (1.13 M) · logistic regression on Grounding DINO scores	A5, C1, C2, C3	**yes: trained (gradient descent)**
The big pretrained models	Grounding DINO 232 M · SAM 2.1 217 M · OpenCLIP 151 M · DINOv2 304 M · MiniLM 22.7 M weights	all	no: frozen (fine-tuning only tried)

Weight counts were read from the loaded models (sum of all parameters) on 5 November 2026.

How to read the coloured labels

Label	Meaning	Example in the app
ZERO-SHOT	The pretrained model is used as it is. We only give it prompts or inputs; nothing is trained.	SAM 2.1 cuts out a pot from a box
FROZEN + TRAINED HEAD	The pretrained model is **frozen** (weights unchanged) and gives a vector. A **small new layer** (projection head, logistic regression) is trained on our data on top.	MiniLM vector → trained 384→256 layer (the colour-match model)
FINE-TUNED	The pretrained model's **own weights are updated** on our data.	only the Grounding DINO experiment
CLASSIC ML	Trained from scratch with classic machine learning, no pretrained network.	TF-IDF + logistic regression, PCA, K-Means
NO MODEL	No learning at all: rules, formulas or a fixed algorithm.	ColorThief, CIEDE2000, typo rules

1. The five pretrained models at a glance

Model (exact checkpoint)	What it is, pretrained on	Output	Used in steps	How
Grounding DINO base *IDEA-Research/grounding-dino-base*	Open-vocabulary object detector: finds boxes for any words you type. Inside: Swin image backbone + BERT text encoder. Pretrained on large image-text detection and grounding data.	boxes + a score per phrase	A1, A5	ZERO-SHOT FINE-TUNED tried, not used
SAM 2.1 large *facebook/sam2.1-hiera-*	Segment Anything 2.1: turns a box into an exact outline (mask). Pretrained on the SA-	3 candidate masks per box,	A2	ZERO-SHOT

large	1B images and SA-V videos.	with predicted quality		
OpenCLIP ViT-B/32 *laion2b_s34b_b79k*	Image-text model trained contrastively on LAION-2B (2 billion image-caption pairs): says how well a picture fits a sentence.	image / text vectors, 512-d	A3, A5	ZERO-SHOT FROZEN + TRAINED HEAD
DINOv2 ViT-L/14 *dinov2_vitl14*	Self-supervised vision transformer (no labels), pretrained on 142 million curated images (LVD-142M). Very good general image features: texture, gloss, shape.	1,024-d vector per object	A5, A6, C2, C3	FROZEN + TRAINED HEAD ZERO-SHOT shape
all-MiniLM-L6-v2 *sentence-transformers/all-MiniLM-L6-v2*	Small sentence-embedding model (6 layers), trained by its authors on over 1 billion sentence pairs. Pinned to one revision so results never change.	384-d vector per text	B2, C1, C2	ZERO-SHOT B2 FROZEN + TRAINED HEAD C1, C2

All five are downloaded once by SETUP.bat into model_cache and then run offline. Exact revisions are pinned in scripts/setup_models.py.

2. The whole flow in one view

Input	Steps	Pretrained models used
📷 **Photo**	A1 find objects → A2 cut out → A3 check ceramic → A4 colours → A5 six attributes → A6 similar pottery	Grounding DINO, SAM 2.1, OpenCLIP, DINOv2
✍️ **Text**	B1 clean tags → B2 text vectors → B3 read free text (card G)	MiniLM
🧪 **Recipe search**	C1 colour match → C2 smart search → C3 the first recipe model	MiniLM, DINOv2
🧮 **Formula**	26 oxides → fractions → square root (C1: + PCA-16) → trained head	none (numbers only)

Activity diagram: from your input to the recipe list

Each box is one step; its colour is how the model is used (legend on the left).

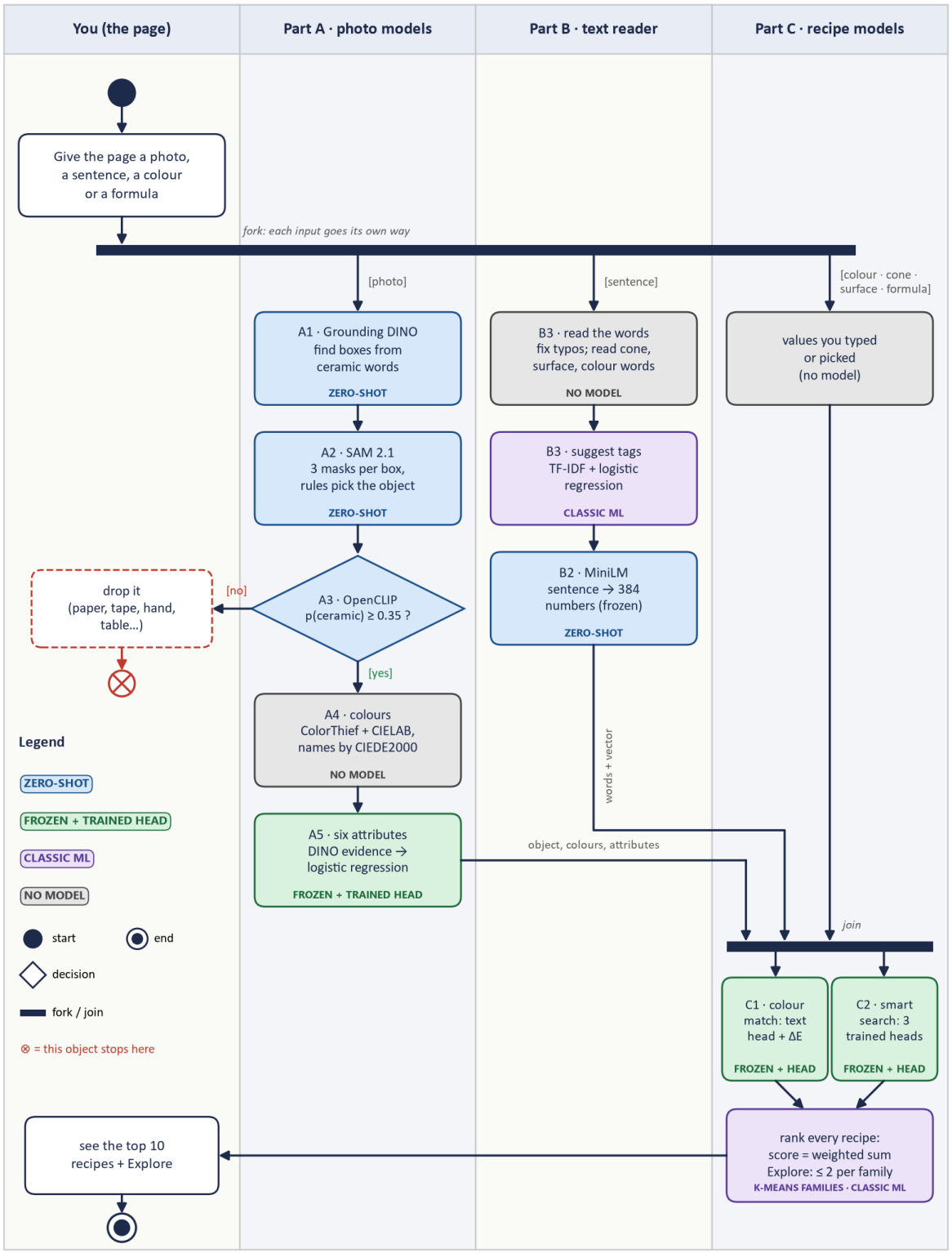


Figure 2 · Activity diagram of one search. Read top to bottom: the fork sends each input to its own lane; the join collects everything for the recipe models. Box colour = how the model is used.

Part A · The ceramic photo

From a photo to separate, measured ceramic objects. The same settings were used for the 7k database photos, so a new photo is measured exactly like the database.

Worked example in every picture below: the app's example photo **577829** (*"Blue bowl in a hand"*) and, for Parts B and C, the sentence *"glossy dark blu celedon with iron speckles, cone 6 oxidation"*.

All numbers are **real outputs** (5 November 2026). The steps inside the segmenter (every box, the 3 SAM candidates, CLIP's score per sentence) were re-run on the CPU with the same code; the final object and the six attributes are the running app's own answer.

A1 · Find the objects

Grounding DINO base ZERO-SHOT

- Prompted with ceramic words; a low threshold first (find everything), filtered later.
- Gives one box per object, plus hand boxes (a hand holding a bowl).

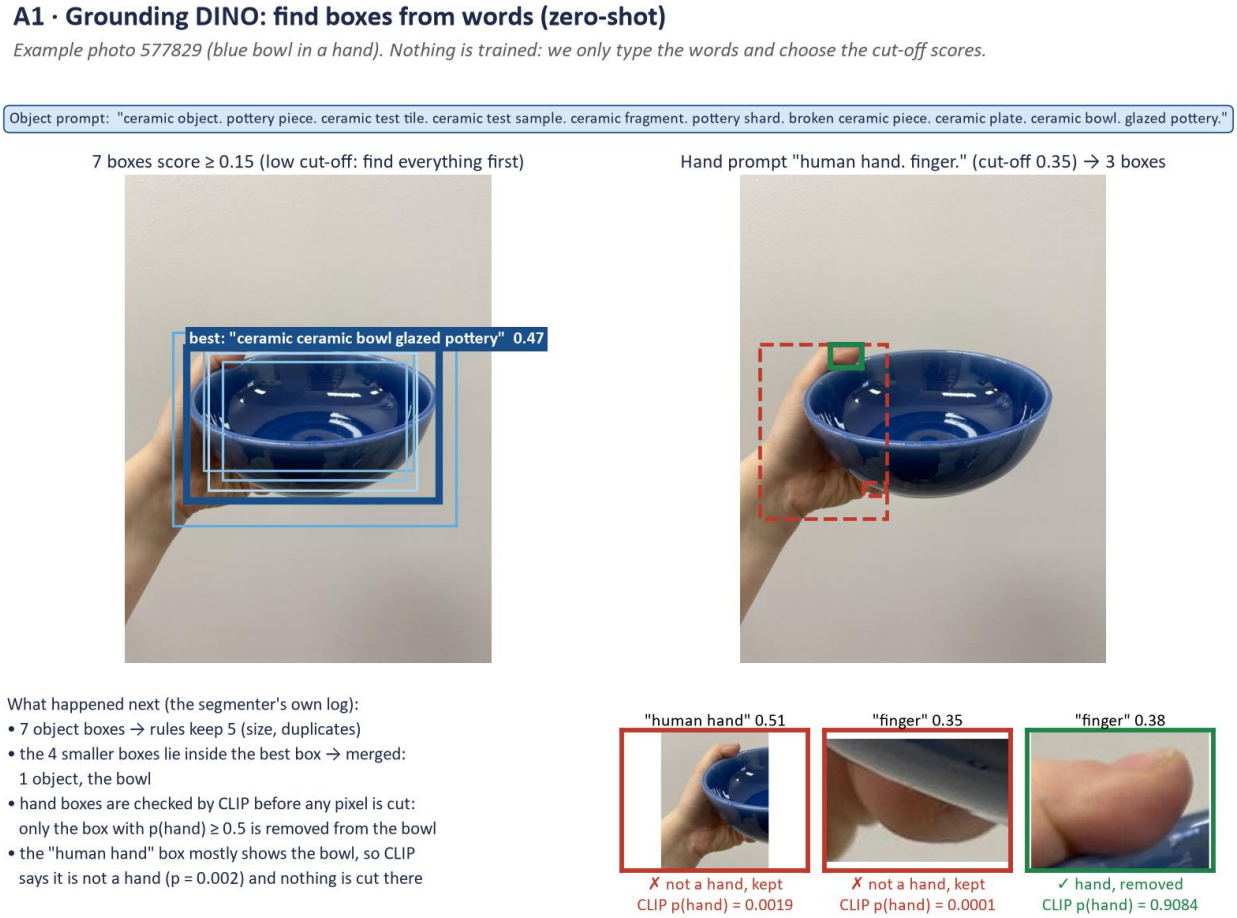


Figure 3 · Grounding DINO on the example. Left: the 7 boxes it finds for the ceramic words. Right: the 3 boxes for "human hand. finger."; CLIP keeps only the real finger (green) as a hand.

A2 · Cut them out

SAM 2.1 large ZERO-SHOT

- The image is encoded once; every box (and one full-frame box) gives 3 candidate masks with SAM's own quality score.
- Rules then choose: one big object or several separate ones, fill holes, split touching tiles.

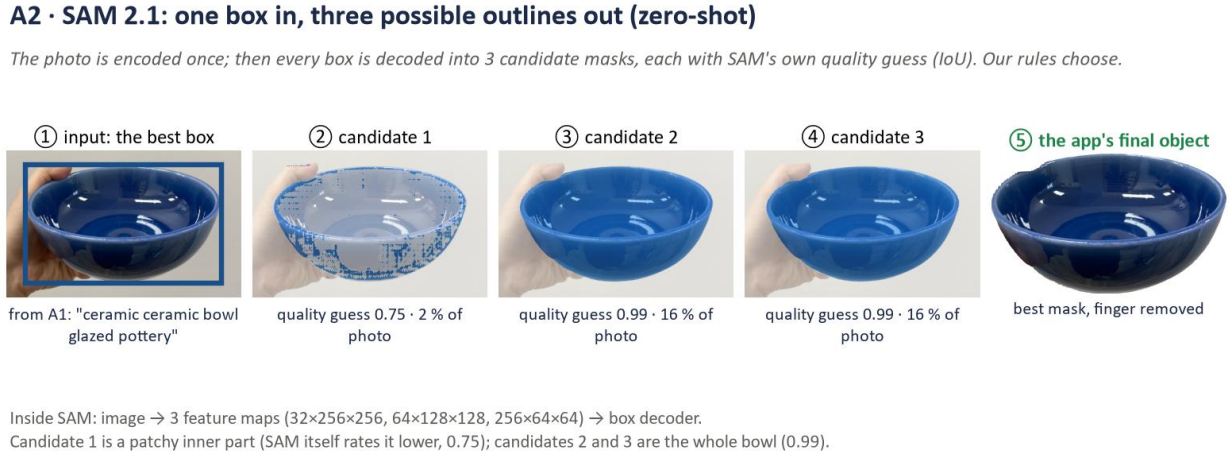


Figure 4 · SAM 2.1 turns the best box into three candidate outlines with its own quality guess; the rules keep the whole bowl and the verified finger is cut away.

A3 · Is it really ceramic?

OpenCLIP ViT-B/32 ZERO-SHOT

- The object is compared with sentences such as *"a glazed ceramic bowl", "a ceramic glaze test tile"* and non-ceramic ones (paper, tape, a hand). A softmax over these sentences gives p(ceramic).
- **p(ceramic) ≥ 0.35** keeps the object; hand boxes are also checked by CLIP before anything is removed.

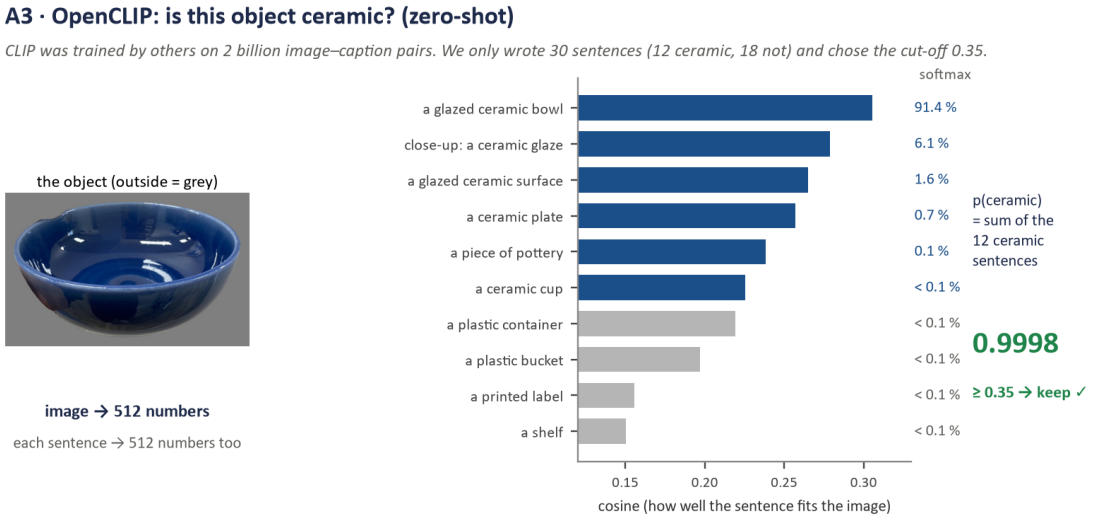


Figure 5 · CLIP compares the cut-out with 30 sentences we wrote. Blue = ceramic sentences, grey = not ceramic. The sum over the ceramic ones is p(ceramic) = 0.9998, so the bowl is kept.

A4 · Colours

NO MODEL no pretrained model

- **ColorThief** (median-cut palette) and k-means in **CIELAB** give the colour regions and their %.
- Names = nearest of the **98 reference shades** by **CIEDE2000** (navy blue, graphite black …).
- Pick colour = median of 13 × 13 pixels inside the object (brightest / darkest 15 % left out).

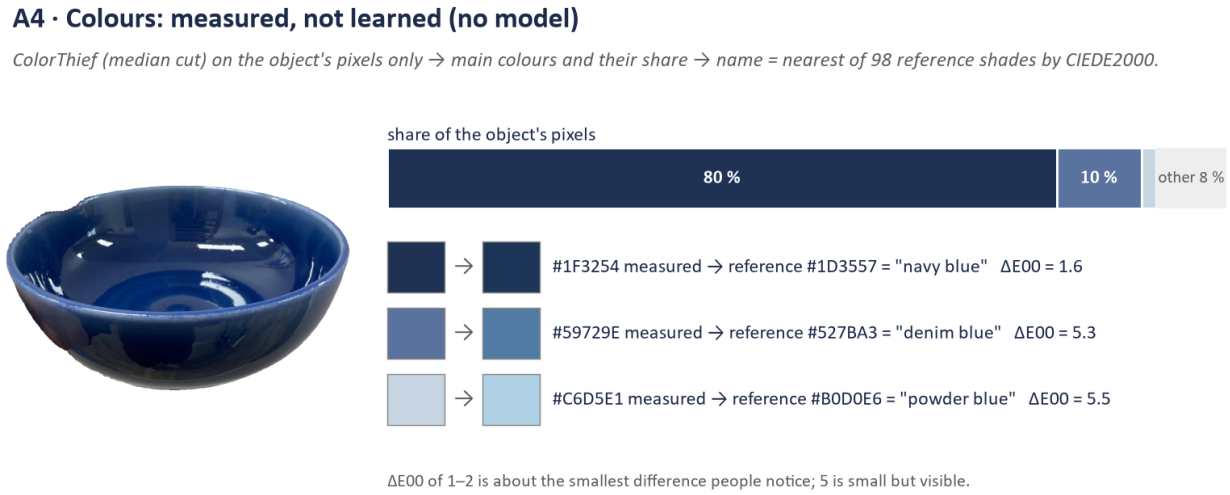


Figure 6 · Colours are measured on the bowl's pixels only, then named by the nearest reference shade (ΔE00 = colour distance). No neural network is involved.

A5 · Six attributes (shape, surface, transparency, texture, effects, defects)

Grounding DINO ZERO-SHOT evidence + **OpenCLIP / DINOv2** FROZEN + TRAINED HEAD features → trained logistic regression

- Grounding DINO is asked one phrase per class (*"glossy glaze . satin glaze . …"*); its scores are the evidence. It stays frozen.
- Earlier heads (v1): per attribute the best on validation of zero-shot CLIP prompts, logistic regression or a small MLP on **frozen CLIP + DINOv2** features.
- **Trained:** an L2 logistic regression turns that evidence into the six attributes, with the top 3 candidates.
  - Data: **1,504** training objects, 240 validation, 240 test (split by recipe group, test scored once).
  - **Labels were made by Claude from the cut-out images, not checked by a person.**
- Experiment: FINE-TUNED fine-tuning Grounding DINO's decoder + box head on the same labels (Swin + BERT kept frozen). **Not used in the app.**

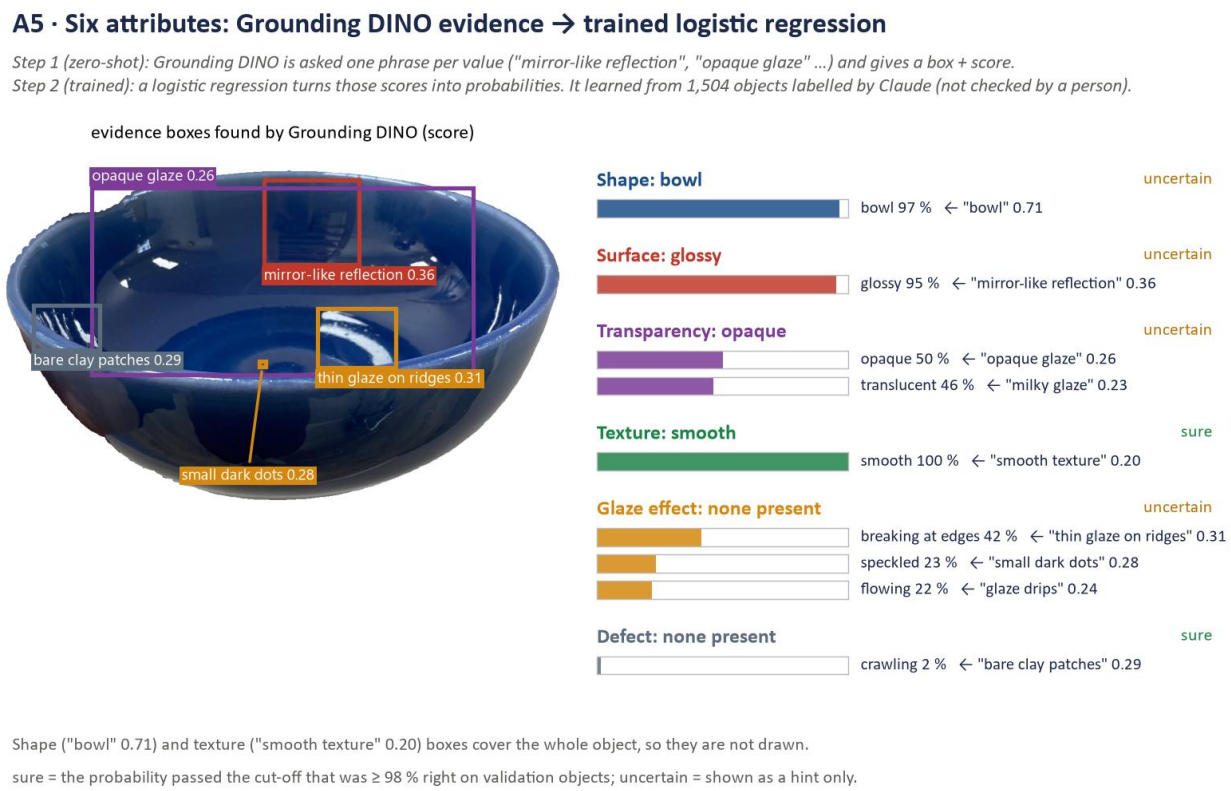


Figure 7 · Left: the evidence boxes Grounding DINO finds for the attribute phrases (zero-shot). Right: the trained logistic regression turns those scores into probabilities, with the top candidates per attribute.

A6 · Similar pottery (object search)

DINOv2 ViT-L/14 FROZEN + TRAINED HEAD

- **Look:** DINOv2 vector of the object (patch tokens averaged inside the mask) → the trained image head of the first recipe model (C3).
- **Shape:** DINOv2 on the black silhouette only (no colour) + outline geometry.
- **Colour:** region colours compared with CIEDE2000 NO MODEL. **Surface:** the attribute probabilities of A5.
- The weights of the four parts were tuned on validation queries (Balanced / Shape / Glaze / Colour).

Part B · The text

How words become something the models can compare. The text branch is in the app as *vendor/gc2_text_branch*.

B1 · Clean the recipe tags

NO MODEL rules

- Glazy tags are normalised (spelling, cone ranges, contradictions flagged, never fixed) and written as one fixed description per recipe (*"Subtype: … Firing cone: …"*).

B2 · Text vectors

all-MiniLM-L6-v2 ZERO-SHOT

- Frozen, pinned revision: **35,461 recipes × 384-d**; the vectors are used directly for a text-only search baseline (cosine similarity + filters).
- The projection head (to 128-d) is built but was **not trained** in the branch; the trained text heads are in C1 and C2.

B2 · What MiniLM knows without any training (zero-shot)

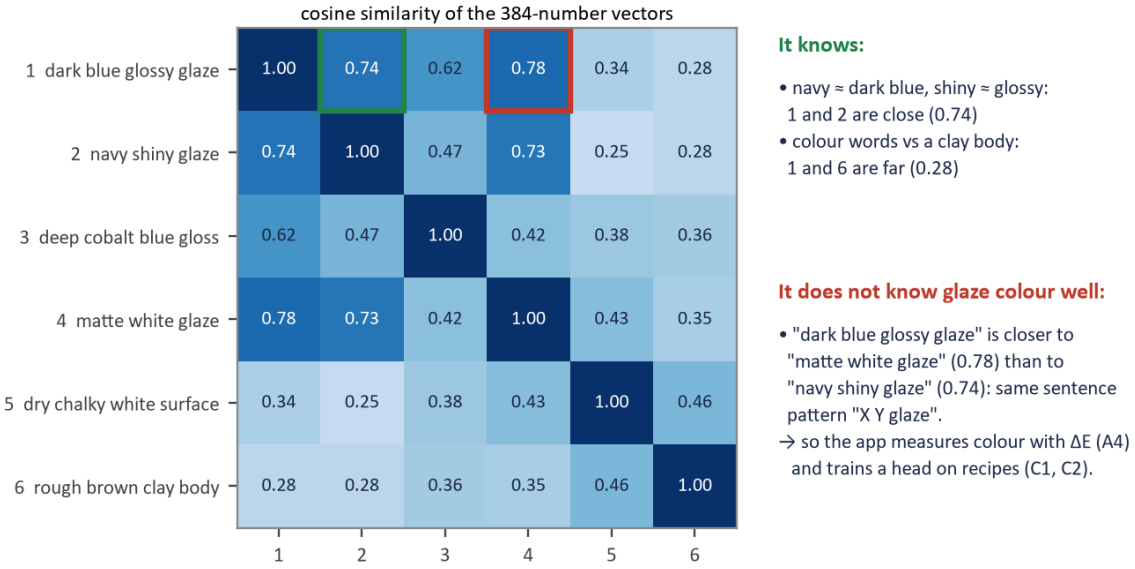


Figure 8 · What MiniLM already knows with no training: similar meanings get similar vectors, but it does not separate glaze colours well. That is why colour is measured (A4) and a head is trained on recipes (C1, C2).

B3 · Read what a person types (card G, Read it)

NO MODEL rules + CLASSIC ML suggestions

- Rules + typo fixing (difflib, ratio ≥ 0.80) read colour, Orton cone (04 ≠ 4), surface, transparency, atmosphere, effects, colourant % and oxides.
- **Suggestions:** TF-IDF (words + character n-grams) → logistic regression, trained on **4,766 Glazy descriptions** (*"a celadon for my gas kiln"* → reduction 81 %).

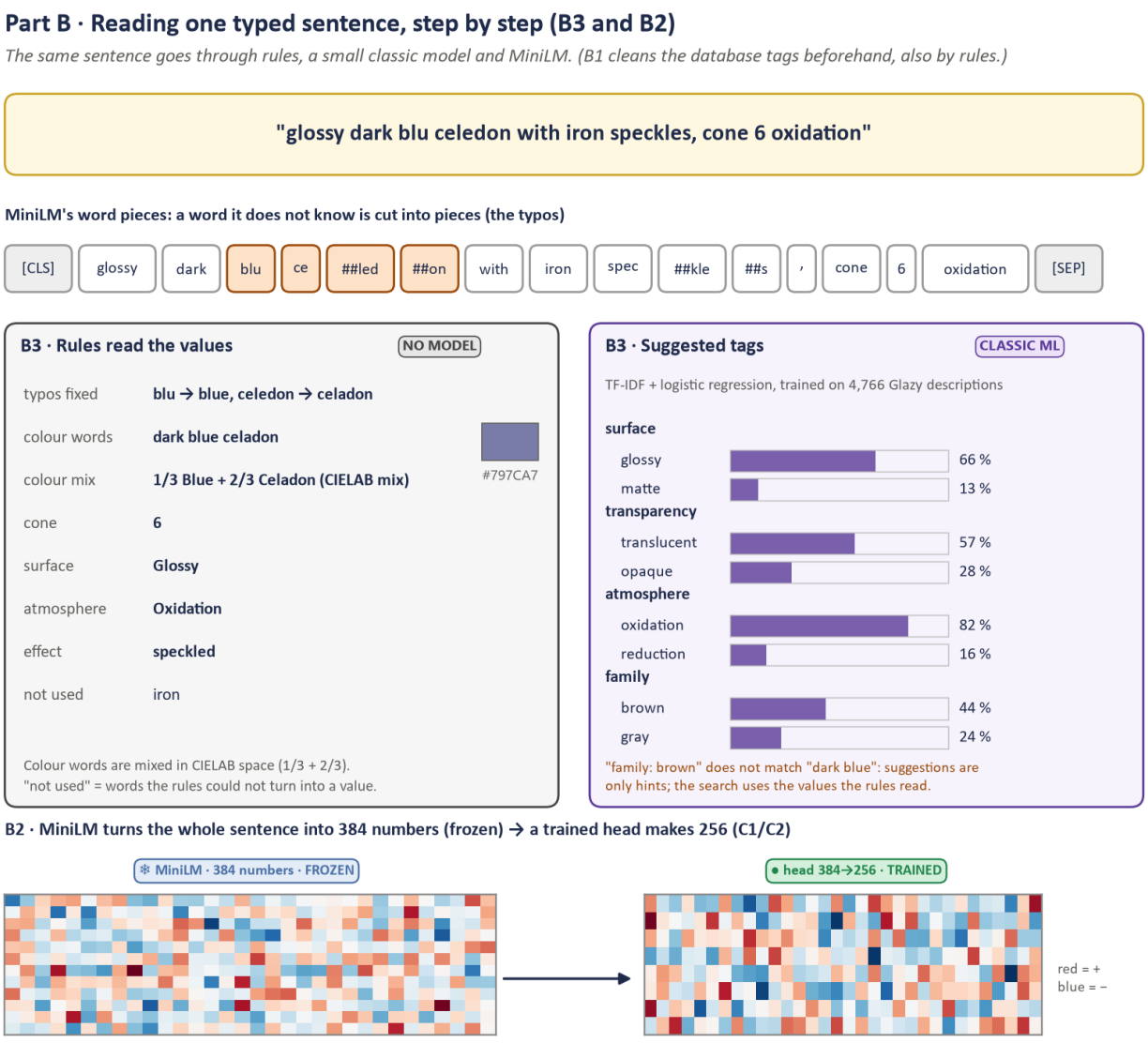


Figure 9 · One typed sentence, step by step: MiniLM's word pieces (typos fall apart), the rules that read the values, the suggested tags, and the 384 numbers MiniLM gives the trained heads.

Finding from the first recipe model (C3): for the short tag text, **TF-IDF + SVD** was selected on validation over frozen MiniLM, CLIP's text tower and a BiLSTM. MiniLM is used where the text is a sentence (C1, C2).

Part C · How the recipe models learn

The common idea: put two kinds of data into **one shared space**. A recipe's text and the same recipe's chemistry (or photo) should land close together, and different recipes far apart.

How it learns: **the pretrained encoders stay frozen; only small projection heads are trained** with **InfoNCE** (contrastive loss): in each batch, the right pair must score higher than all the wrong pairs (softmax over the batch). Recipes of the same composition group are not used as wrong pairs.

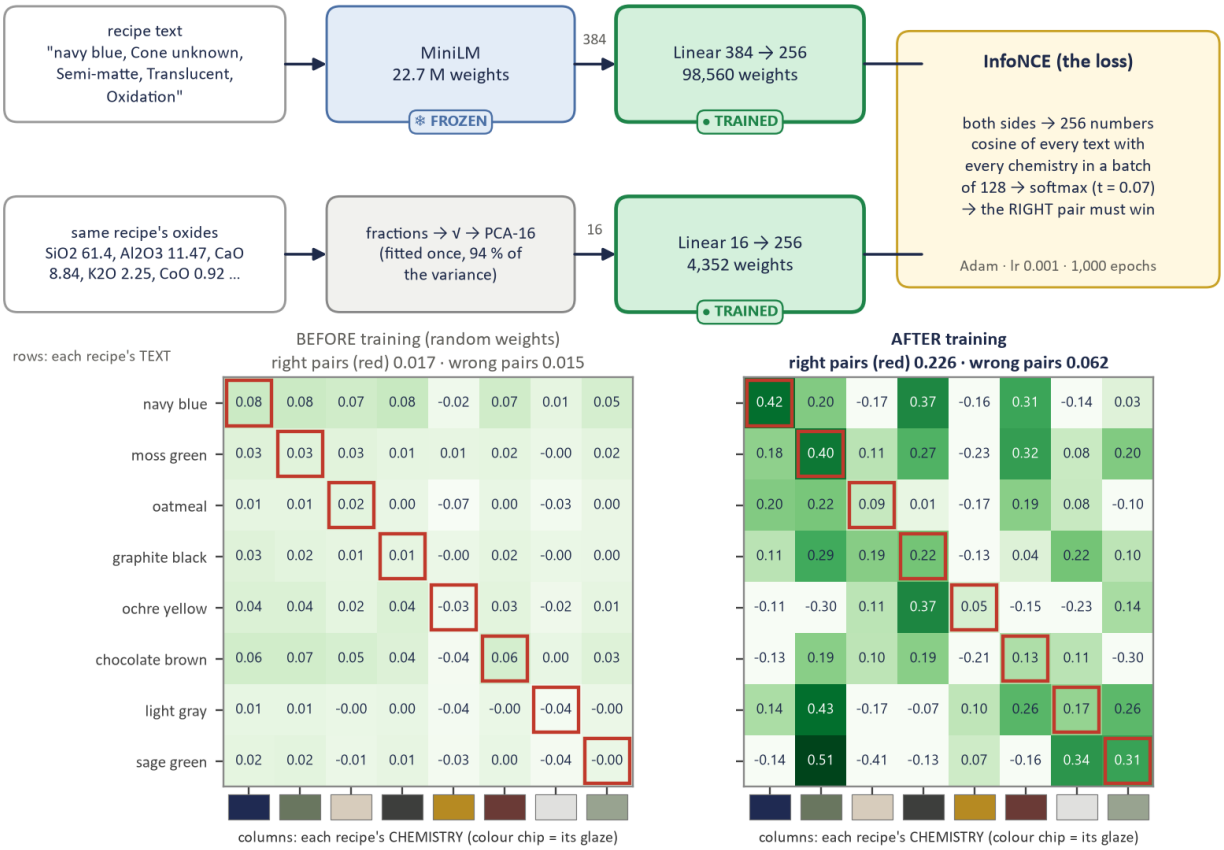
C1 · Colour match model (card G, Colour match)

all-MiniLM-L6-v2 FROZEN + TRAINED HEAD + chemistry CLASSIC ML

- **Text side:** *"color: Blue, Cone 6, Glossy, unknown, unknown"* → frozen MiniLM (mean pooled, 384-d) → **trained** Linear 384 → 256.
- **Chemistry side:** 26 oxides → fractions → square root → **PCA-16** (fitted on training recipes only) → **trained** Linear 16 → 256.
- **Training:** symmetric masked InfoNCE, temperature 0.07; Adam, learning rate 0.001, batch 128, 1,000 epochs, seed 1.
    - Split by **composition groups** (agglomerative clustering, cosine distance 0.01), 80 / 20; training recipes almost identical to a test recipe (cosine > 0.99) are dropped (no leakage).
- **Families for Explore:** K-Means on the PCA features, k chosen by silhouette (5 – 20).
- **Score:** 0.40 learned match + 0.40 colour (CIEDE2000) + 0.10 surface + 0.10 cone. In the app: same lists as the notebook (scores within 0.00001).

C1 · How the colour-match model learns: only two small layers are trained

Real numbers from the trained model; the 8 recipes are held-out test recipes (never seen in training).



On all 1,309 test recipes: right pair 0.238 vs wrong pairs 0.076 after training, 0.036 vs 0.037 with random weights.
It learns families of glazes more than single recipes, and some labels are noisy, so the right recipe is not always the darkest cell in its row.
That is why the app also adds the measured colour (ΔE), cone and surface to the score.

Figure 10 · How the colour-match model learns. Top: only the two green layers are trained. Bottom: the same 8 test recipes before and after training; training pulls each recipe's text towards its own chemistry (red cells).

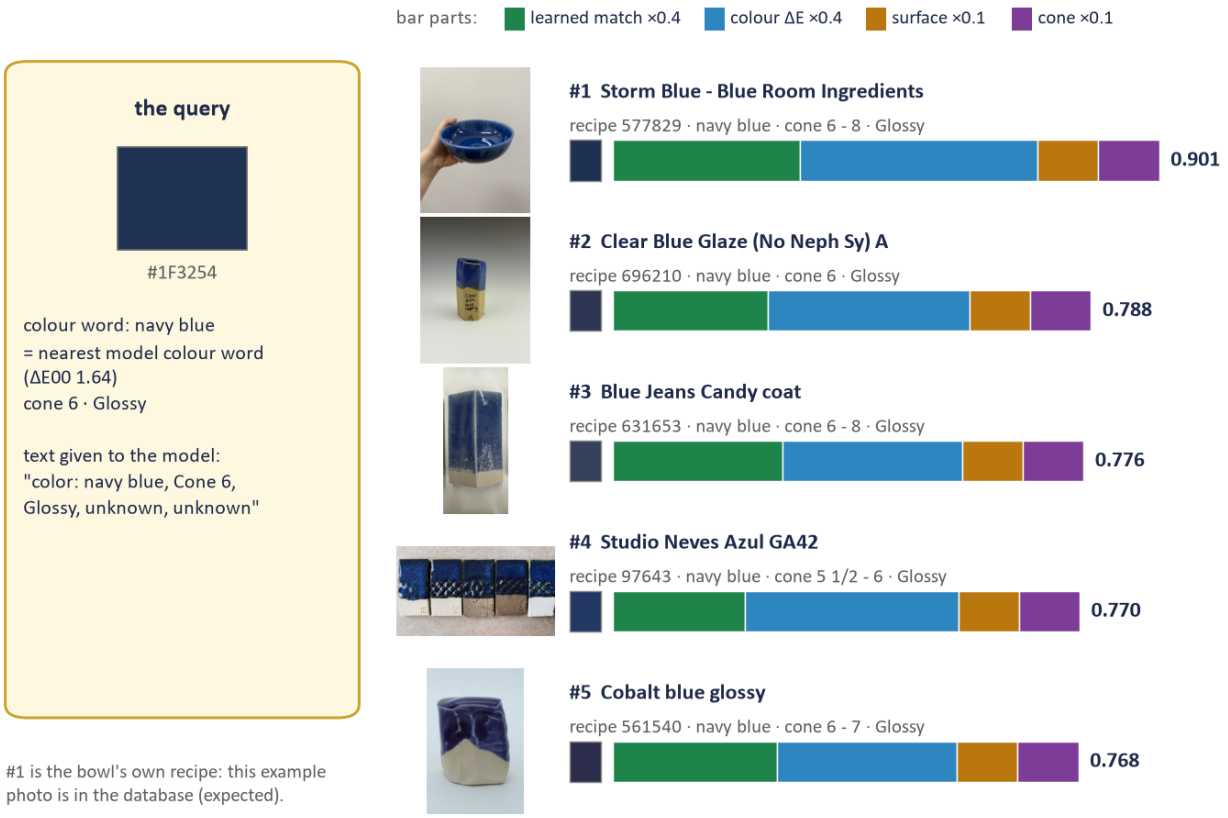


Figure 11 · The bowl's measured colour sent to colour match: the five best recipes and how each score is built from its four parts.

C2 · Smart search: the joint model (card G, Smart search)

MiniLM FROZEN + TRAINED HEAD + **DINOv2** FROZEN + TRAINED HEAD + formula NO MODEL

- Extends the same idea to **three** inputs: text (MiniLM, 384-d), photo (DINOv2, 1,024-d), formula (26 oxides, √).
- **Trained:** three heads (Linear → GELU → Dropout 0.1 → Linear, hidden 512, output 256-d).
- **Training:** InfoNCE for text–photo, text–formula and photo–formula (temperature 0.07); AdamW 0.001, weight decay 0.0001, batch 256, early stop on validation (best epoch 58 of 70).
  - Data: 7,063 recipes split 4,945 / 1,059 / 1,059 by composition group. Text per step: tags 25 %, the Glazy description 20 %, a noisy sentence with typos 55 %.
- Result: with noisy typed text, the right recipe is in the **top 10 for 74.2 %** of test recipes.
- **Families:** K-Means on the 256-d embeddings, k = 10 by silhouette (DBSCAN / HDBSCAN tried: one big cluster).

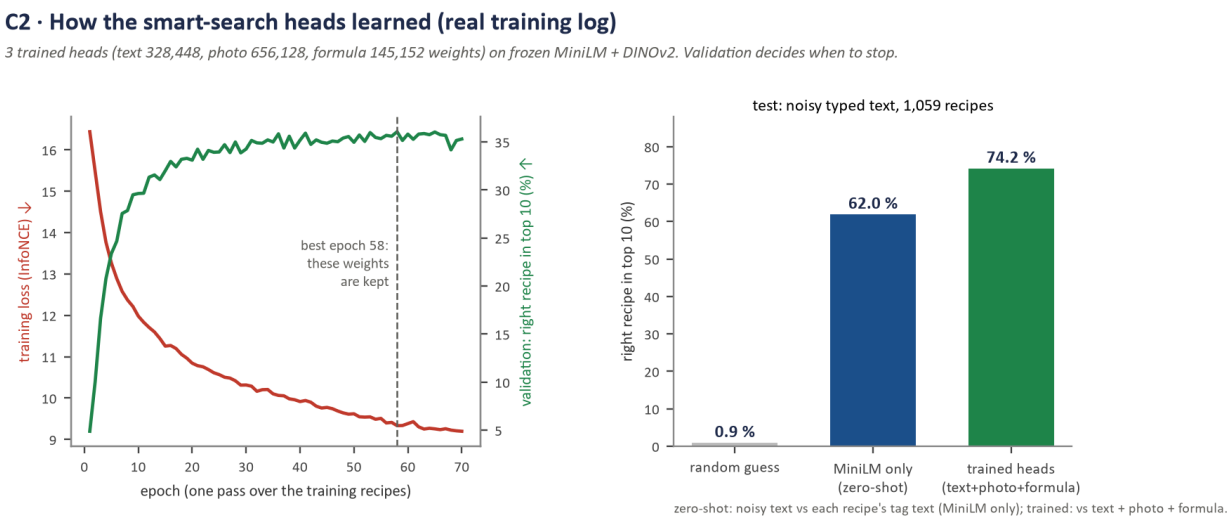


Figure 12 · The smart-search heads learning: the loss goes down, the validation score goes up and levels off; the weights of the best epoch are kept. Right: the trained heads beat MiniLM alone on the same noisy typed text.

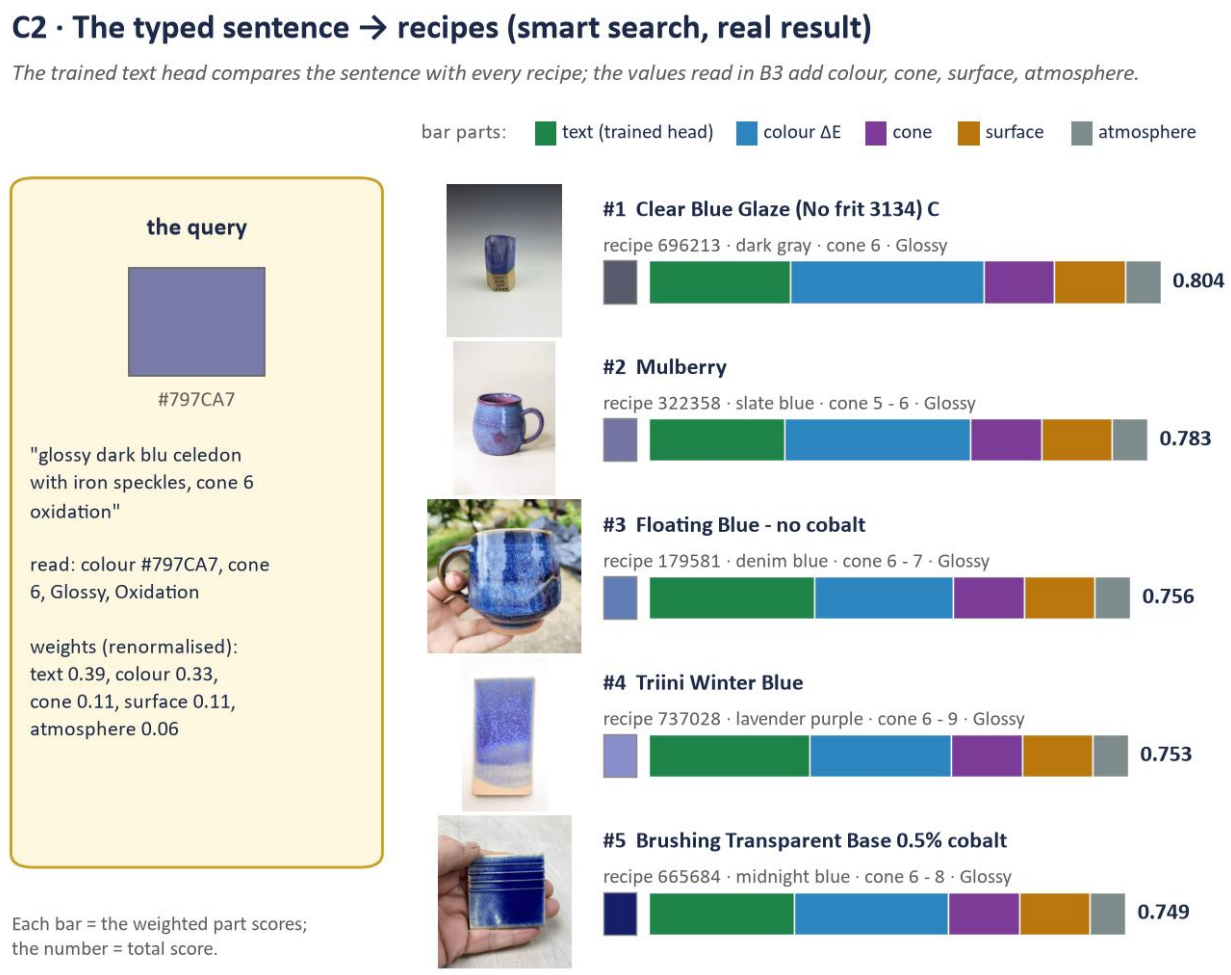


Figure 13 · The example sentence sent to smart search: the five best recipes, each score split into the text model and the values read from the sentence.

C3 · The first recipe model (Recipes (previous model) search, and the look part of A6)

DINOv2 FROZEN + TRAINED HEAD + TF-IDF CLASSIC ML + formula CLASSIC ML

- Image: frozen DINOv2 (masked pooling). Text: TF-IDF + SVD (256-d). Formula: UMF oxides + ingredient SVD (95-d).
- **Trained:** one MLP head per input (hidden 512 → 256-d), InfoNCE temperature 0.1, AdamW 0.001, batch 512, early stop (patience 12). Fusion: image 0.40, text 0.35, formula 0.25.

The formula-similarity part (MCS metric) is not in this repository, so its models are not listed here.

3. Zero-shot or fine-tuned? One table

Pretrained model	ZERO-SHOT	FROZEN + TRAINED HEAD	FINE-TUNED
Grounding DINO base	A1 find objects · A5 attribute evidence	A5 logistic regression on its scores	experiment only (not in the app)
SAM 2.1 large	A2 cut-out masks	-	-
OpenCLIP ViT-B/32	A3 is it ceramic?	A5 first attribute heads	-
DINOv2 ViT-L/14	A6 silhouette (shape)	A5 · A6 look · C2 photo · C3 image	-
all-MiniLM-L6-v2	B2 text-only search (text branch)	C1 text · C2 text	-

In the app, the fine-tuned column is empty.

Why not fine-tune? Our labelled data is small (1,504 attribute objects with machine-made labels; about 5,000 training recipes). Fine-tuning a large network on that risks over-fitting and losing what it already knows. Frozen features + small heads train in minutes on an 8 GB GPU (or the CPU), and each head can be retrained without touching the others.

4. What was trained, and where the files are

Trained part	Kind	File (in multimodal_experiments)
C1 colour match heads	FROZEN + TRAINED HEAD	models/glaze_retrieval/shared_space_heads.pt
C2 smart search heads + families	FROZEN + TRAINED HEAD CLASSIC ML	models/glaze_joint/heads.pt, gallery.npz
B3 text suggestions	CLASSIC ML	models/glaze_joint/text_tags.joblib
C3 first recipe model	FROZEN + TRAINED HEAD	work/results/final_model.pt
A5 six attributes	FROZEN + TRAINED HEAD CLASSIC ML	work/attributes_v3/attr_v3_models.joblib
A6 search weights	tuned	work/attributes_v1/balanced_weights.json

5. Read more

- **docs/glaze_integration_architecture.md**: card G, both engines, every setting.
- **docs/manuals/CG2_Glaze_Search_Process_Models_MindMap.pdf**: the process with the model of each step.
- **docs/manuals/CG2_Glaze_Search_Manual.pdf**: how to use the app, with real examples.
- Code: *glazeprep/segmentation/ceramic_v3.py* (A1–A3), *mm/attr_v3.py* (A5), *app/object_search.py* (A6), *mm/glaze_text.py* (B3), *mm/glaze_retrieval.py* (C1), *mm/glaze_joint.py* (C2), *mm/model.py* (C3).